INSTITUTO J&F · GERMINARE
PROJETO INTERDISCIPLINAR · 2º ANO TECH

RELATÓRIO GERENCIAL

BUSINESS INTELLIGENCE · NEXUS

Gerenciamento inteligente de despensa e combate ao desperdício de alimentos

DATABRICKS · BRONZE-SILVER-GOLD | AI/BI DASHBOARD | JOB DIÁRIO | 9 KPIs

Autoria: Bruna Oliveira — Frente de Business Intelligence do Projeto NexUs

Disciplina: Business Intelligence — Prof. Daniel Freitas

Versão: 2.0 — novembro de 2026, com evidências do dashboard e do Job

Sumário

1 Resumo executivo

2 Contextualização e problema de negócio

3 Base de dados e arquitetura

4 Tratamento e transformação dos dados

5 Indicadores de negócio (KPIs)

6 Pipeline de dados no Databricks

7 Jobs e automação — evidências reais

8 Dashboard de Business Intelligence — evidências reais

9 Gerador de relatórios e consumo externo

10 Resultados e insights

11 Tecnologias utilizadas e por quê

12 Limitações e próximos passos

13 Anexos — checklist de evidências e código-fonte

> **Nota de elaboração:** todo número e toda imagem deste documento vêm de arquivos reais do repositório ou de capturas de tela do workspace do Databricks enviadas pela autora em 02/11/2026. Nenhum dado foi estimado ou decorativo.

1 · RESUMO EXECUTIVO

O NexUs é um aplicativo de gerenciamento de despensa que permite cadastrar lotes de alimentos com validade, definir estoque mínimo e registrar consumo ou descarte de cada item. Esta frente de Business Intelligence transforma esses registros operacionais em uma base analítica confiável, calcula indicadores padronizados de desperdício e risco de estoque, e os entrega em um dashboard interativo — automatizando leitura, tratamento, cálculo e atualização diária dos dados.

PANORAMA ATUAL (DASHBOARD, 02/11/2026)

D1 · DESPERDÍCIO	D2 · ESQUECIMENTO	D3 · VENCIDOS	D4 · VENCEM 7D
11,30%	38,61%	14,97%	2,64%

D6 · ABAIXO MÍNIMO	D7 · VIDA ÚTIL (D)	D8 · SEM VALIDADE	D9 · CATEGORIA LÍDER
68,46%	169	3,86%	Frutas/Verduras

Item	Síntese
Base analisada	3.175 lotes de estoque, 9.590 movimentos de consumo/descarte, 660 perfis (520 simulados para fins de BI), 590 alimentos, em 27 UFs
Indicadores	9 KPIs (D1 a D9), calculados por **unidade de produto (embalagem)**, com meta, desvio e situação
Pipeline	1 Job com 2 tasks (ingestão Bronze + pipeline Silver/Gold), rodando automaticamente todo dia às 07:00 — evidência de 9 execuções diárias consecutivas em anexo
Dashboard	3 páginas, 12 datasets, cards, gráficos de barra/linha, histogramas, boxplots, mapa e filtros interativos

⚠ **Principal achado:** a taxa geral de desperdício está dentro da meta (D1 = 11,3%), mas o **esquecimento** — descarte muito tempo após o vencimento (D2 = 38,6%) — responde por boa parte do volume perdido. O problema não é falta de aviso de "está vencendo"; é o item ser esquecido depois de já ter vencido. Recomendação central: lembretes de validade dentro do app.

SEÇÃO 2

CONTEXTUALIZAÇÃO E PROBLEMA DE NEGÓCIO

O NEXUS E O PAPEL DO BI

O NexUs ajuda famílias (e também empresas e estabelecimentos) a controlar o que têm em casa: o que foi comprado, quando vence, o que foi consumido e o que foi jogado fora. Isoladamente, cada registro é só um dado operacional. O papel do BI é agregar milhares desses registros e transformá-los em indicadores que respondem perguntas de gestão: estamos desperdiçando mais ou menos que no mês passado? Por que as pessoas descartam comida? Onde o problema é mais grave? O BI não substitui o app — é a camada que olha para o conjunto de dados e devolve inteligência sobre o comportamento de uso.

O PROBLEMA SEM ANÁLISE ESTRUTURADA

Sem uma camada analítica, os dados ficam presos em tabelas transacionais, otimizadas para o app responder rápido a uma tela — não para responder "quanto desperdiçamos esse mês". Seria preciso rodar consultas pesadas direto no banco de produção, sem histórico consolidado, sem regras de cálculo padronizadas e sem visualização acessível a quem não lê SQL. Decisões de produto (vale investir em alertas de validade?) ficariam baseadas em intuição, não em dado.

As perguntas que o BI responde

#	Pergunta de negócio	Indicadores envolvidos
1	Quanto se desperdiça, e isso está melhorando?	D1 e sua evolução mensal (média móvel, variação)
2	Por que o alimento vai para o lixo?	D2 e o momento do descarte
3	O que se desperdiça?	D9 (por categoria, Pareto) e D7 (vida útil)
4	Onde está o risco hoje?	D3, D4, D5 (por UF) e D8, com mapa geográfico
5	A casa tem o que precisa?	D6 (itens abaixo do estoque mínimo configurado pelo usuário)

Natureza da base: dados simulados

O NexUs ainda não possui usuários reais em produção. A base combina a massa de **seed** padrão (perfis Faker, origem `APP`) com uma **carga de simulação dedicada ao BI** (origem `SIMULADO_BI`): 520 perfis, ~520 alimentos e 12 meses de histórico, com padrões de comportamento propositalmente programados (Frutas/Verduras e Padaria desperdiçam mais; a região Norte tem desperdício acima da média; quem usa o app há mais tempo desperdiça menos). O objetivo não é apresentar descobertas sobre um público real, e sim demonstrar a **capacidade da solução de detectar esses padrões** — o que o dashboard faz com sucesso (seção 10).

3 · Base de dados e arquitetura

3.1 ORIGEM DOS DADOS

O banco operacional do NexUs é um **PostgreSQL hospedado no Neon**. Além do schema `public` (tabelas transacionais), o banco mantém um schema `bi` com **19 views analíticas** já modeladas em estrela — construídas na frente de Arte e Modelagem do projeto — e o schema histórico `bi_hist.registro_estoque`, com cada operação de consumo e descarte. MongoDB (dados desorganizados) e Redis (cache sem histórico) ficam **fora do escopo** do BI.

3.2 COMO OS DADOS CHEGAM AO DATABRICKS

O Databricks não copia o banco manualmente: usa **Lakehouse Federation**, conectando diretamente ao Neon e expondo-o como um catálogo a mais. Foi criada uma *connection* (`neon_nexus`) com um usuário **somente leitura** (`databricks_bi`), que vira o catálogo estrangeiro `nexus_pg`. Qualquer notebook ou pipeline lê `nexus_pg.bi.vw_kpi_cards` como se fosse tabela local — nunca com permissão de escrita em produção.

3.3 ESTRUTURA OPERACIONAL × ESTRUTURA ANALÍTICA

A **estrutura operacional** (tabelas do app, normalizadas, rápidas para 1 tela por vez) é diferente da **estrutura analítica** (modelo em estrela: dimensões descrevendo "quem/o quê/onde" e fatos registrando "o que aconteceu", otimizado para agregar milhares de linhas de uma vez). As 19 views `bi.*` já fazem essa primeira tradução no Postgres; o Databricks lê essas views e aplica uma segunda camada de modelagem — a arquitetura medalhão — até a base final.

3.4 MIGRATION V11 E O NOVO MODELO DE EVENTOS

Uma migration de banco (V11) reestruturou `bi_hist.registro_estoque`: o evento passou a referenciar o lote (`pantry_item_id`), com tipo `CONSUMED`/ `DISCARDED` e quantidade sempre > 0. Consequências tratadas na Silver:

- Um lote pode ter **várias operações parciais** — por isso os KPIs são por unidade, não por operação (seção 5.0);
- O descarte pode acontecer **antes** do vencimento — tratado separadamente no "momento do descarte".

Lote
1 linha de estoque. "8 pães que vencem dia 22" é 1 lote com quantidade 8 — não 8 lotes.

Unidade
1 embalagem (1 pacote, 1 garrafa). Métrica oficial dos KPIs de desperdício.

Estado
"Foto" da despensa num dia. Base de D3, D4, D5, D6, D7, D8.

Evento
Histórico do que saiu, com data própria. Base de D1, D2 e D9.

3.5 VOLUMES DA BASE (PRODUÇÃO)

Entidade	Quantidade	Observação
Lotes na despensa	3.175	186 do seed/testes + 2.989 da carga de simulação de BI
Movimentos de consumo/descarte	9.590	12 meses de histórico simulado (2025-11 a 2026-10)
Perfis	660	520 de origem `SIMULADO_BI` (380 casas, 55 empresas, 85 lojas)

Alimentos cadastrados	590	catálogo com categoria
Config. de estoque mínimo	2.368	2.226 com mínimo > 0 (D6)

A modelagem estrela e as 19 views do banco foram construídas na frente de Arte e Modelagem de Dados e são preservadas sem alterações como evidência dessa disciplina. A frente de BI consome essas views como fonte e constrói, por cima, uma segunda modelagem própria dentro do Databricks, orientada aos KPIs (seção 6.3).

4 · Tratamento e transformação dos dados

Entre a cópia bruta das views do Neon (Bronze) e a base usada pelo dashboard (Gold), os dados passam pela camada **Silver**, responsável por limpeza, tipagem e regras de qualidade.

4.1 PROBLEMAS IDENTIFICADOS E TRATAMENTO

Problema	Tratamento
Linha duplicada	`ROW_NUMBER() OVER (PARTITION BY chave ORDER BY _ingerido_em DESC);` mantém a gravação mais recente
Datas como `AAAAMMDD`, `-1` = sem data	`try_to_date(...)` → `DATE`; `-1` vira `NULL`
Tipos genéricos da federação	`CAST` para `DECIMAL/INT`
Textos com espaço/caixa/vazios	`TRIM`, `UPPER/LOWER`, `NULLIF`; vazio vira "Não informado"
Chave órfã	Vira `-1`; cada dimensão tem linha sentinela `-1 = Não informado`
UF inválida/ausente	Validada contra tabela fixa `ref_uf`; fora da lista vira `N/I`
"Hoje" congelado no momento da extração	`dias_ate_vencer/status_validade` recalculados com a data de cada foto
2 vocabulários de evento (pré/pós V11)	`CASE` padroniza para `CONSUMED/DISCARDED`
Geolocalização	Lat/long do endereço não são trazidas (dado pessoal); mapa usa a capital da UF

4.2 REGRAS DE QUALIDADE (EXPECTATIONS)

Tabela	Descarta a linha	Só alerta
`dim_alimento`	`sk_alimento` nula	alimento sem nome
`dim_localidade`	`sk_localidade` nula	UF `N/I`
`fato_estoque`	`id_item` / entrada nulos	quantidade ≤ 0; validade antes da entrada
`fato_movimento`	id/tipo/data nulos ou inválidos	evento antes da compra; sem quantidade

Resultado observado: alertas em `fato_estoque` (8 lotes com cadastro inconsistente, mantidos e sinalizados) e 1 cidade com UF inválida — nenhuma linha perdida indevidamente.

4.3 EXEMPLO REAL — REGRA DE NEGÓCIO DERIVADA (GOLD)

O "momento do descarte" não existe como coluna no banco de origem — é calculado comparando a data do evento com a data de validade:

```
CASE WHEN m.expirado = 0                          THEN NULL
     WHEN m.data_validade IS NULL                 THEN '4 · Sem validade'
     WHEN m.data_evento <  m.data_validade        THEN '1 · Antes de vencer'
     WHEN DATEDIFF(m.data_evento, m.data_validade) <= 7 THEN '2 · Até 7 dias após vencer'
     ELSE '3 · Esquecido (+7 dias após vencer)' END AS momento_descarte
```

SEÇÃO 5

INDICADORES DE NEGÓCIO (KPIS)

5.0 POR QUE CALCULAR POR UNIDADE, E NÃO POR REGISTRO

Na primeira versão dos KPIs, contar **operações** em vez de **unidades** (embalagens) distorcia o resultado. Exemplo real do projeto: um usuário que consome 10 pacotes em 1 operação e descarta 1 pacote em outra teria, contando por registro, desperdício de 50% (1 em 2 operações) — quando na prática só **9,09%** do que ele movimentou (1 de 11 unidades) foi para o lixo. Com a V11, em que um lote pode ter diversas baixas parciais, isso fica ainda mais grave. Por isso, **todos os 9 indicadores são oficialmente calculados por unidade**; a versão por operação/lote foi mantida como complemento (`valor_por_lote`) só para conferência com a view do banco (que conta operações).

> **EVENTO** indicador sobre o histórico de consumo/descarte (tem data, filtra por mês) · **ESTADO** indicador sobre a foto da despensa de hoje (histórico diário desde 25/10/2026).

D1 Taxa de desperdício — EVENTO

Objetivo	Do que saiu da despensa, qual fração virou lixo em vez de ser consumida.
Fórmula	`Σ unid. descartadas ÷ Σ unid. que saíram (consumidas + descartadas)`
Fonte	`gold.fato_movimento`, grão de unidade (expode por quantidade)
Meta / resultado	≤ 15% · **11,30%** · dentro da meta
Decisão apoiada	Priorizar investimento em redução de desperdício (alertas, sugestões de consumo)
Onde aparece	Card na Visão geral; linha de evolução mensal + média móvel 3 meses

D2 Descarte por esquecimento — EVENTO

Objetivo	Separar descarte logo após vencer de descarte de item esquecido por muito tempo.
Fórmula	`Σ unid. descartadas +7 dias após vencer ÷ Σ unid. descartadas com validade`
Meta / resultado	sem meta · **38,61%**
Decisão apoiada	Justifica lembretes/notificações de validade, não "comprar menos"
Onde aparece	Barras "Momento do descarte", página Desperdício

D3 Itens vencidos na despensa — ESTADO

Fórmula	`Σ unid. vencidas ÷ Σ unid. com validade (saldo > 0)`
Meta / resultado	≤ 5% · **14,97%** · fora da meta
Interpretação	Retrato do agora: itens vencidos pendentes de baixa
Onde aparece	Card na Visão geral; histograma de dias até vencer, página Despensa

D4 Itens vencendo em 7 dias — ESTADO

Fórmula	`Σ unid. com 0≤dias até vencer≤7 ÷ Σ unid. com validade`
Meta / resultado	≤ 10% · **2,64%** · dentro da meta
Decisão apoiada	Base para notificação "consuma logo" dentro do app

D5 Risco por UF — ESTADO

Fórmula	`% unid. vencidas/vencendo ≤30d por UF; ranking DENSE_RANK() sobre o %`
Regra de amostra	UFs com menos de 30 lotes excluídas do ranking
Resultado	UF nº 1 do card: **33,33%** (Rondônia) · sem meta
Onde aparece	Mapa de pontos e ranking de barras, página Despensa

D6 Estoque abaixo do mínimo — ESTADO

Fórmula	`casas com ≥1 item abaixo do mínimo ÷ casas com mínimo configurado`
Regra de negócio	Item vencido não conta como estoque (mesma regra do app)
Meta / resultado	≤ 30% · **68,46%** · fora da meta
Decisão apoiada	Sustenta a proposta de lista de compras automática

D7 Vida útil no cadastro — ESTADO

Fórmula	`mediana de (validade − entrada), ponderada por quantidade, em dias`
Resultado	**169 dias** · sem meta
Onde aparece	Boxplot de vida útil por categoria, página Despensa

D8 Itens sem validade — ESTADO

Fórmula	`Σ unid. sem validade ÷ Σ unid. totais`
Meta / resultado	≤ 10% · **3,86%** · dentro da meta
Decisão apoiada	Tornar o preenchimento de validade mais simples no cadastro

D9 Desperdício por categoria — EVENTO

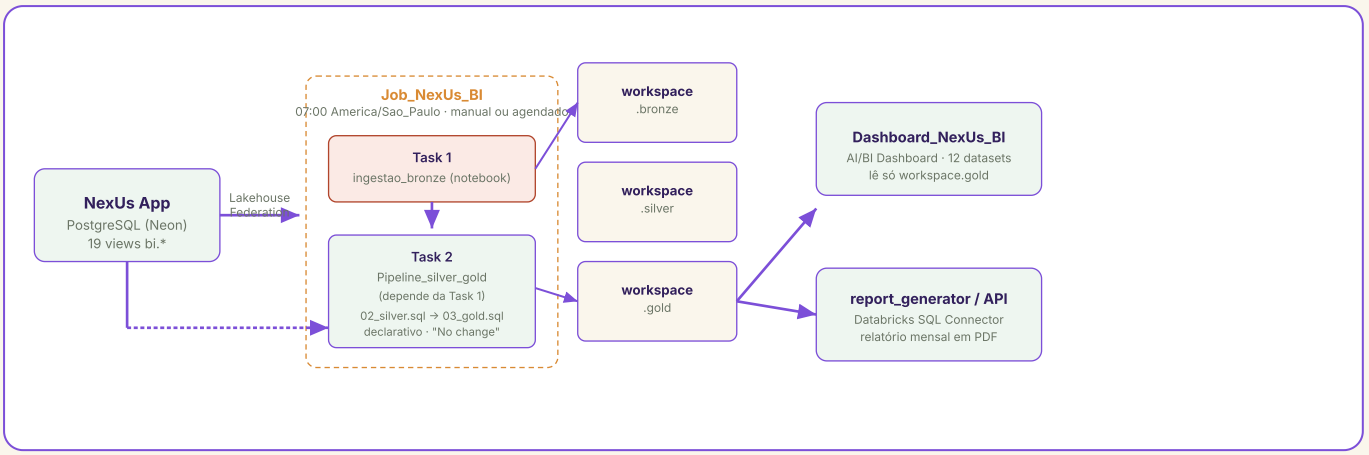
Fórmula	`Σ unid. descartadas da categoria ÷ casas que tiveram a categoria`
Análise	Curva de Pareto: % acumulado do desperdício por categoria
Resultado	Categoria líder: **Frutas, Verduras e Legumes**

Os nove indicadores estão **implementados e em produção** na camada Gold — não há KPI apenas planejado.

SEÇÃO 6

PIPELINE DE DADOS NO DATABRICKS

6.1 FLUXO COMPLETO



App (Neon) → Lakehouse Federation → Job (Task 1 Bronze, Task 2 Silver+Gold) → `workspace.bronze/silver/gold` → consumo pelo Dashboard e pelo gerador de relatório/API.

6.2 BRONZE — A "FOTO DO DIA"

Copia 7 views do Neon sem tratamento, com `_data_referencia` (data de Brasília), `_ingerido_em` e `_origem`. Ingestão **idempotente por dia** (`DELETE+INSERT` na data de hoje) — rodar 2x não duplica. Fixa `SET TIME ZONE 'America/Sao_Paulo'` para evitar virada de dia prematura (Neon roda em GMT).

6.3 SILVER E GOLD

Silver: 8 tabelas de limpeza. Gold: 16 tabelas — star schema (`dim_*`, `fato_*`) + tabelas `kpi_*` prontas. **Nenhuma tabela do dashboard lê Bronze ou Silver diretamente** — garantia de que toda visualização parte de dado já tratado. O pipeline é **declarativo**: resolve sozinho a ordem Silver → Gold e recalcula só o que mudou.

SEÇÃO 7

JOBS E AUTOMAÇÃO — EVIDÊNCIAS REAIS

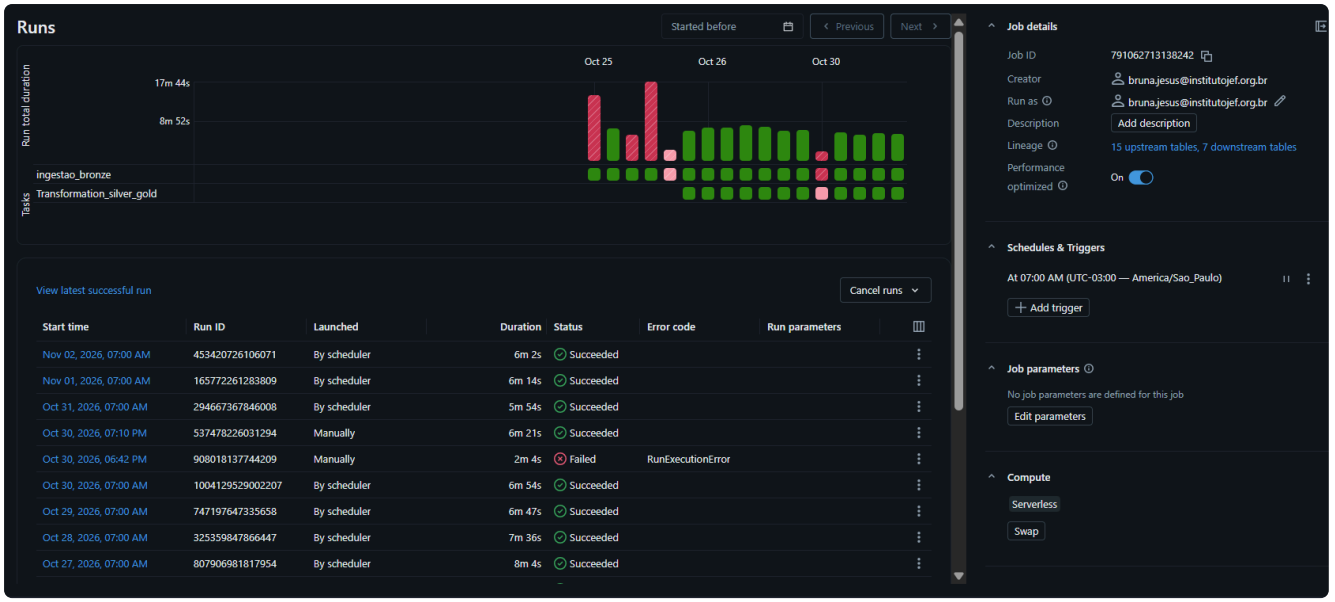
O Job `Job_NexUs_BI` encapsula todo o fluxo (ingestão + transformação) em uma execução orquestrada, disparável manualmente (*Run now*) ou agendada — sem rodar célula a célula. Está configurado com 2 tasks: `ingestao_bronze` (notebook) → depende dela → `Transformation_silver_gold` (pipeline ETL).

EXECUÇÃO REAL — 02/11/2026, DISPARADA PELO AGENDAMENTO



O que mostra: a execução do dia 02/11/2026, disparada automaticamente pelo agendamento (*Launched: By scheduler*), rodou das 07:00 às 07:06 com status **Succeeded** — `ingestao_bronze` em 1min46s e `Transformation_silver_gold` em 4min13s, total **6min01s**. O grafo confirma a dependência entre as duas tasks e o painel lateral mostra 54 queries executadas e 649.167 linhas lidas naquela execução.

HISTÓRICO DE EXECUÇÕES E AGENDAMENTO



O que mostra: o agendamento ativo **"At 07:00 AM (UTC-03:00 — America/Sao_Paulo)"** e o histórico de execuções diárias consecutivas de 27/10 a 02/11/2026, todas com status **Succeeded** e disparadas *By scheduler* — prova de que a automação está rodando sozinha, dia após dia, sem intervenção manual. O gráfico de barras também preserva, de forma transparente, duas execuções que falharam durante o desenvolvimento (em vermelho, 30/10), mantidas como evidência do processo de depuração relatado na seção 6.

LIÇÕES DE CONFIGURAÇÃO REGISTRADAS NO HISTÓRICO

- Um arquivo de consulta avulsa deixado na pasta de transformações do pipeline quebrava a inicialização inteira — regra adotada: conferências sempre no SQL Editor;
- Uma tentativa inicial de pipeline separado para a Gold falhou com "tabela já gerenciada por outro pipeline" — corrigido unificando Silver e Gold no mesmo pipeline.

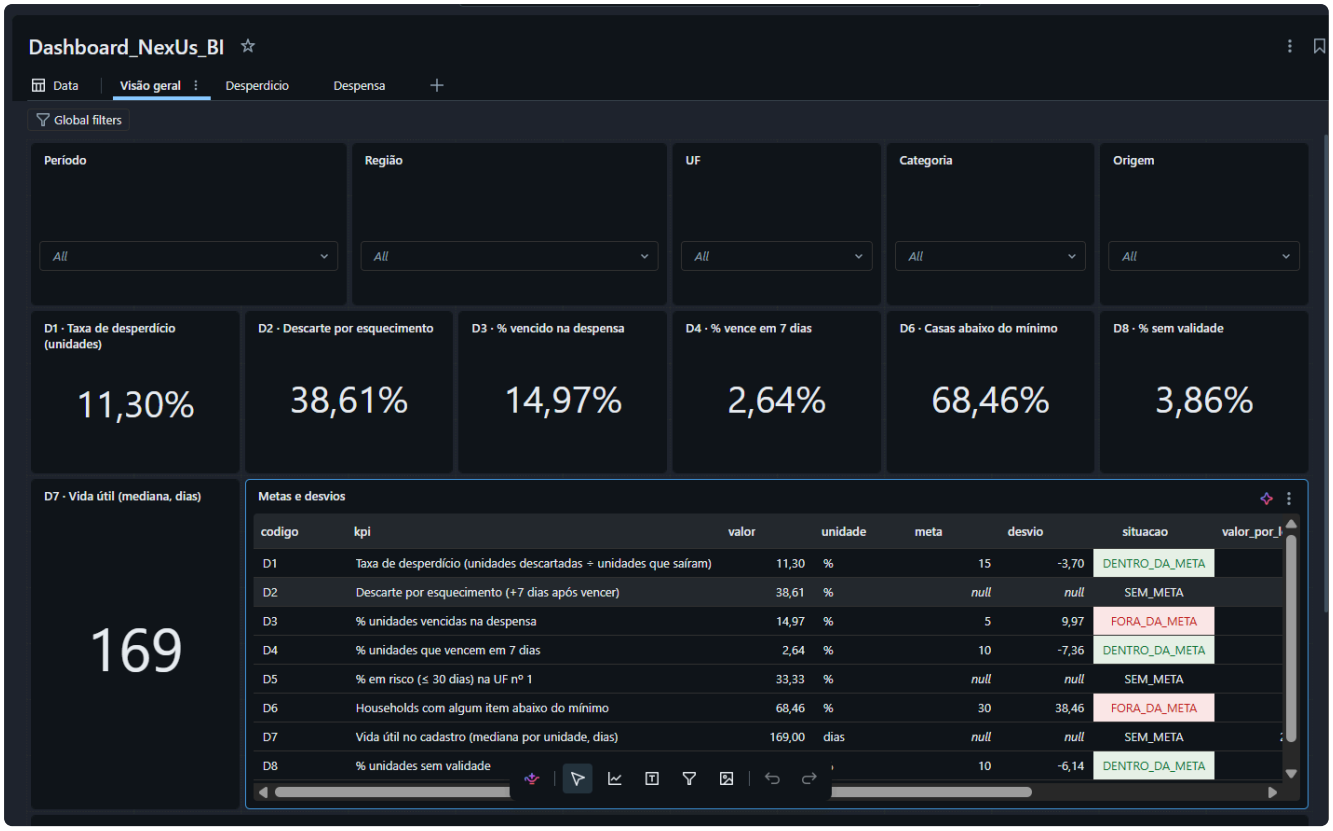
Evidência ainda pendente: print da aba *Tasks* do Job (configuração estática das 2 tasks) e do grafo interno do `Pipeline_silver_gold` (Silver → Gold, aba *Graph* do pipeline) — complementam, mas não substituem, o que já está comprovado acima.

SEÇÃO 8

DASHBOARD DE BI — EVIDÊNCIAS REAIS

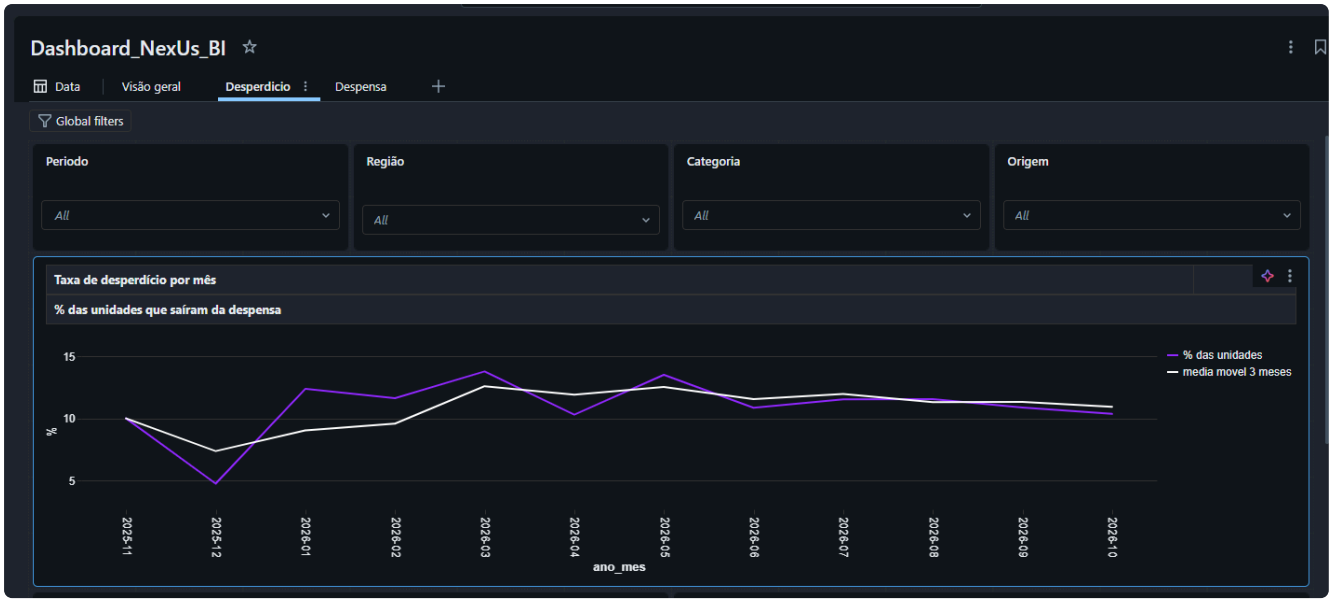
`Dashboard_NexUs_BI` é um AI/BI Dashboard nativo do Databricks, com **12 datasets SQL** lendo exclusivamente de `workspace.gold`, organizados em 3 páginas: **Visão geral**, **Desperdício** e **Despensa**. Dois datasets usam `LATERAL VIEW explode(sequence(1, quantidade))` para expandir 1 linha por lote em 1 linha por unidade — assim `AVG()` sobre uma coluna 0/1 já é o KPI oficial, e os filtros interativos (Período, Região, UF, Categoria, Origem) funcionam sobre qualquer subconjunto dos dados.

PÁGINA 1 · VISÃO GERAL



O que mostra: 7 cards de KPI (D1, D2, D3, D4, D6, D7, D8), 5 filtros interativos (Período, Região, UF, Categoria, Origem — aqui todos em "All", sem filtro aplicado) e a tabela "Metas e desvios" com o valor, a meta, o desvio em pontos percentuais e a situação de cada indicador. **Leitura:** D1, D4 e D8 estão dentro da meta; D3 (14,97%) e D6 (68,46%) estão fora — é exatamente o padrão descrito no resumo executivo: o fluxo de desperdício está controlado, mas o estoque parado (itens vencidos e abaixo do mínimo) é o ponto de atenção.

PÁGINA 2 · DESPERDÍCIO — EVOLUÇÃO MENSAL DE D1



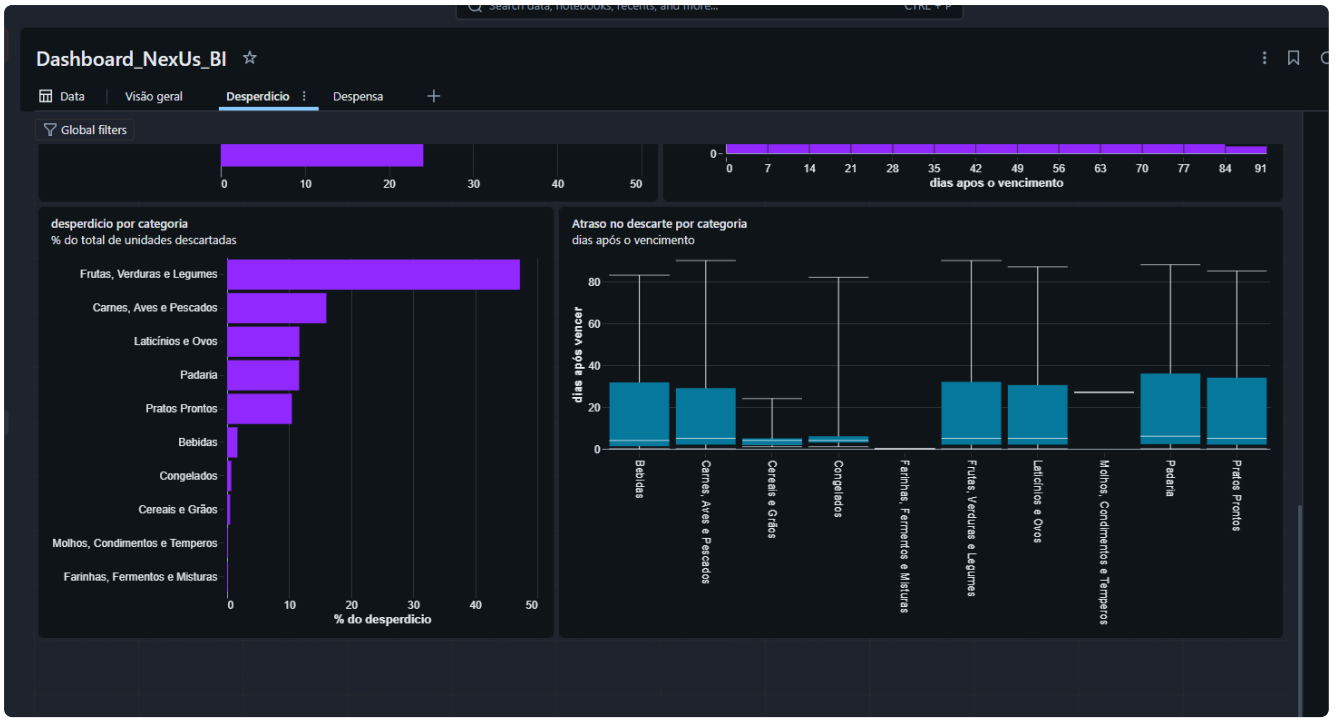
O que mostra: a taxa de desperdício mensal (linha roxa) e sua média móvel de 3 meses (linha branca), de novembro/2025 a outubro/2026. **Leitura:** a taxa oscila mês a mês — com picos em janeiro, março e maio de 2026 — mas a média móvel suaviza essas variações e mostra uma tendência de estabilização entre 10% e 11% a partir da segunda metade de 2026, dentro da meta de 15%.

MOMENTO DO DESCARTE E ATRASO EM RELAÇÃO AO VENCIMENTO



O que mostra: à esquerda, a distribuição do descarte por momento — confirma visualmente o D2: a maior fatia (~47%) é descartada até 7 dias após vencer, mas uma fatia quase tão grande (~29%) é "esquecida" por mais de 7 dias, e ~24% não tinha validade cadastrada. À direita, o histograma mostra a concentração de descartes logo nos primeiros dias após o vencimento, com uma cauda longa se estendendo até ~90 dias — são esses casos de cauda longa que formam o D2.

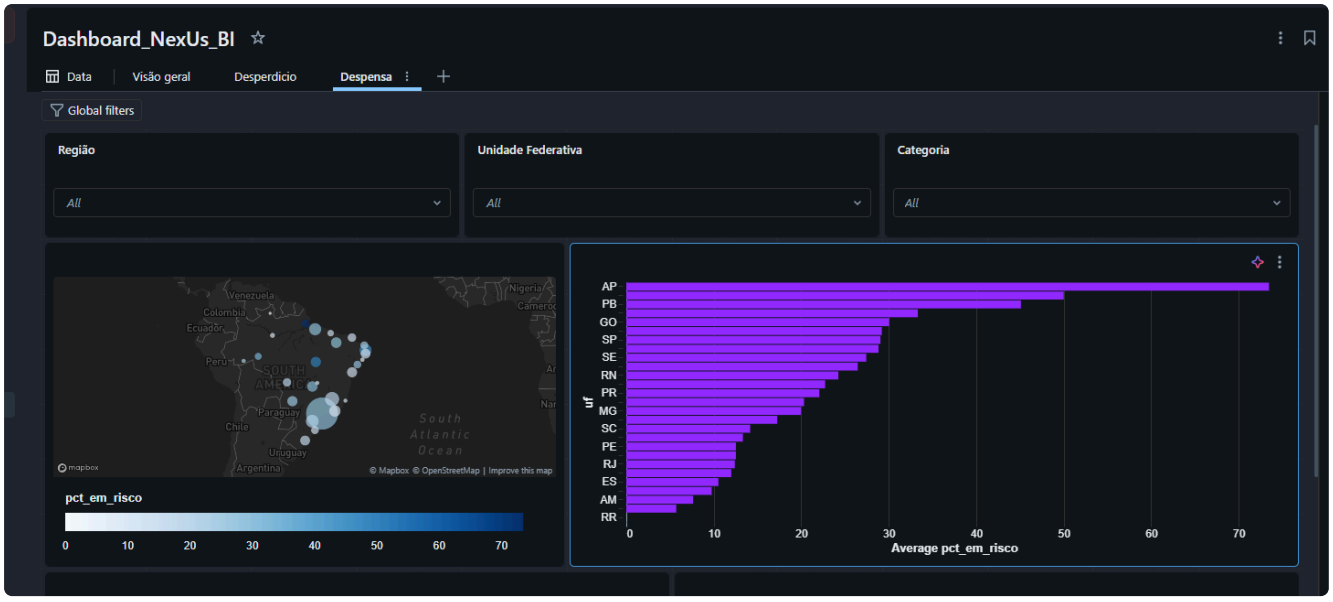
DESPERDÍCIO E ATRASO POR CATEGORIA (PARETO)



O que mostra: à esquerda, o ranking de categorias por % do total descartado — **Frutas, Verduras e Legumes** lidera isoladamente, seguida por Carnes/Aves/ Pescados, confirmando o D9. À direita, o boxplot de dias de atraso por categoria mostra que várias categorias têm valores extremos (outliers) acima de 60-80 dias de atraso, reforçando que o esquecimento prolongado não é um caso isolado, mas se repete em categorias diferentes.

⚠️ **Ponto a verificar antes da apresentação:** o card D9/categoria aponta "Frutas, Verduras e Legumes" como líder, consistente com este gráfico — ok. Já a página Despensa (próxima página) mostra o Amapá no topo do ranking de risco por UF com percentual bem mais alto que os 33,33% do card D5 "UF nº 1". Vale conferir no SQL Editor se o filtro `lotes >= 30` de `ds_uf` está mesmo sendo aplicado nessa visão antes de apresentar — não alterei nada aqui, só registro a divergência para você decidir.

PÁGINA 3 · DESPENSA — MAPA E RISCO POR UF (D5)



O que mostra: mapa de pontos coloridos pelo `pct_em_risco` (quanto mais escuro, maior o risco) e o ranking de barras do D5 por UF. O gráfico evidencia forte concentração geográfica do risco — poucas UFs concentram os maiores percentuais, o que confirma o insight de que o problema pode ser endereçado de forma regionalizada, não genérica.

STATUS DE VALIDADE, DIAS ATÉ VENCER E VIDA ÚTIL POR CATEGORIA



O que mostra: barras empilhadas de status de validade por categoria (OK, vencido, vence em 7/30 dias, sem validade), histograma de dias até o vencimento com pico logo após 0 e cauda longa até ~900 dias, e boxplot de vida

útil mediana por categoria — a base do D7 (mediana geral de 169 dias), mostrando grande variação entre categorias como esperado (itens de mercearia duram muito mais que hortifrúti).

9 · Gerador de relatórios e consumo externo

Além do dashboard interativo, o projeto prevê um gerador de relatório gerencial mensal em PDF e uma API de BI, consumidos por uma aplicação web desenvolvida em paralelo. Ambos leem a camada **Gold** — nunca Bronze ou Silver — via *Databricks SQL Connector*, reutilizando as mesmas tabelas `kpi_*` da seção 6, garantindo que o número do relatório seja sempre o mesmo número do dashboard.

9.1 EVENTO × ESTADO NO RELATÓRIO MENSAL

D3–D8 representam a **foto de hoje** da despensa, não um histórico por mês. Por isso o relatório distingue:

- **Indicadores de evento** (D1, D2, D9) — recalculáveis para qualquer mês de `fato_movimento.ano_mes_evento` (novembro/2025 a outubro/2026);
- **Indicadores de estado** (D3–D8, D5 por UF) — só existem a partir de 25/10/2026; para meses anteriores, o relatório mostra um aviso explícito em vez de reaproveitar a foto atual.

9.2 CONTROLE DE CUSTO COMPUTACIONAL

Como o workspace está na Free Edition (cota diária de computação), o gerador de relatório mantém **cache local** e consulta o warehouse no máximo duas vezes por dia (recomendado: 08:00 e 20:00).

SEÇÃO 10

RESULTADOS E INSIGHTS

KPI	Valor oficial	Situação
D1 — Taxa de desperdício	11,30%	dentro da meta (≤15%)
D2 — Descarte por esquecimento	38,61%	sem meta
D3 — % vencido na despensa	14,97%	fora da meta (≤5%)
D4 — % vence em 7 dias	2,64%	dentro da meta (≤10%)
D5 — Risco por UF (card, UF nº1)	33,33% — Rondônia	sem meta
D6 — Abaixo do mínimo	68,46% das casas	fora da meta (≤30%)
D7 — Vida útil mediana	169 dias	sem meta
D8 — % sem validade	3,86%	dentro da meta (≤10%)
D9 — Categoria líder	Frutas, Verduras e Legumes	sem meta

Valores lidos diretamente do dashboard em 02/11/2026 (cards e tabela "Metas e desvios" da página Visão geral).

10.1 INSIGHTS

1. **O problema central é o esquecimento, não a validade curta.** 38,6% do descarte com validade cadastrada acontece mais de 7 dias após o vencimento. *Recomendação: lembretes de vencimento dentro do app.*
2. **Ausência de validade também gera desperdício evitável.** ~24% das unidades descartadas não tinham data informada — o app não tinha como avisar. *Recomendação: simplificar o preenchimento da validade no cadastro.*
3. **Contar por registro escondia metade do problema.** Medir por unidade (em vez de por lote/operação) revelou que são as compras em maior quantidade que mais vencem. *Recomendação: alertar ao cadastrar quantidades grandes de itens perecíveis.*
4. **D6 é alto por casa, mas o problema é pontual.** A maioria das casas tem algo a repor, mas a fração de itens efetivamente abaixo do mínimo é bem menor. *Recomendação: lista de compras automática, de alto valor percebido e baixo esforço.*
5. **O risco está no estoque parado, não no fluxo.** D1 dentro da meta, mas D3 e D6 fora — o desperdício agregado parece controlado, mas há itens específicos acumulando risco.
6. **Concentração geográfica e por categoria.** O mapa/ranking por UF e a curva de Pareto por categoria mostram que poucas regiões e categorias concentram a maior parte do problema, permitindo ação direcionada em vez de genérica.

Os padrões acima foram programados na massa de simulação (seção 2); o valor demonstrado é a **capacidade do pipeline e do dashboard de detectá-los corretamente**, não uma descoberta sobre usuários reais.

11 · Tecnologias utilizadas e por quê

Tecnologia	Por que foi escolhida
Databricks Free Edition	Plataforma unificada de engenharia e BI, sem custo, com Jobs, pipelines declarativos e AI/BI Dashboard nativos
Lakehouse Federation	Consulta o Postgres de produção sem copiar credenciais no código nem abrir acesso de escrita
Lakeflow Declarative Pipelines	Resolve a ordem de dependência, aplica qualidade declarativa, evita recomputar o que não mudou
Arquitetura medalhão	Separa dado cru de dado confiável; permite auditar cada etapa isoladamente
Spark SQL (*window functions*, `LATERAL VIEW explode`)	Suporte nativo a médias móveis, ranking e Pareto sem código procedural
AI/BI Dashboard	Dashboard nativo do mesmo workspace, filtros interativos ligados direto à Gold
Databricks SQL Connector (Python)	Permite que web app e relatório leiam a Gold programaticamente

12 · Limitações e próximos passos

LIMITAÇÕES

- **Dados simulados:** o app ainda não tem usuários reais.
- **Sem estimativa financeira:** o app não registra preço de compra — nenhum valor em R$ é calculado.
- **Histórico de estado desde 25/10/2026:** D3–D8/UF/dia só existem a partir da primeira execução diária do Job.
- **MongoDB e Redis fora do escopo:** dados não estruturados/cache de curta duração.
- **Fuso da view V11:** pequenas diferenças na conferência em torno da virada do dia são esperadas, não indicam erro no pipeline.
- **Cota da Free Edition:** execução diária única e cache do relatório existem para respeitar a cota gratuita.

PRÓXIMOS PASSOS

- Integrar o app real ao fluxo de consumo/descarte, substituindo gradualmente a massa simulada;
- Finalizar API de BI e gerador de relatório mensal;
- Avaliar uma 3ª task no Job para replicar `kpi_*` em um schema de leitura no Neon;
- Com dados reais, revisitar as metas de cada KPI.

13 · Anexos — checklist de evidências e código-fonte

#	Evidência	Status
1	Configuração das 2 tasks do Job	confirmar na aba Tasks
2	Execução Succeeded, com tempos	anexado (seção 7)
3	Agendamento 07:00 America/Sao_Paulo	anexado (seção 7)
4	Histórico de execuções (com disparo agendado)	anexado (seção 7)
5	Grafo interno do pipeline (Silver → Gold)	pendente
6	Resultado da `kpi_conferencia`	pendente
7	Lista dos 12 datasets do dashboard	pendente
8	Página 1 — Visão geral (sem filtro)	anexado (seção 8)
9	Página 1 — Visão geral (com filtro ativo)	pendente
10	Página 2 — Desperdício (completa)	anexado (seção 8)
11	Página 3 — Despensa (completa)	anexado (seção 8)

ARQUIVOS-FONTE DO PIPELINE

Arquivo	Camada	Função
`databricks/ingestao/01_ingestao_bronze.sql`	Bronze	Notebook da Task 1 do Job
`databricks/pipeline/02_silver.sql`	Silver	Limpeza e padronização (8 tabelas)
`databricks/pipeline/03_gold.sql`	Gold	Star schema + KPIs (16 tabelas)
`databricks/dashboard/datasets.sql`	Consumo	12 datasets SQL do AI/BI Dashboard

Código versionado no repositório do projeto (branch `feature/bi-databricks`), com documentação própria em cada pasta.